1. Problem Title & Link

Title: LeetCode 26 : Remove Duplicates from Sorted Array

Link: LeetCode 26 - Remove Duplicates from Sorted Array

2. Problem Statement (Short Summary)

You are given a **sorted array**:

nums[]

Remove duplicates **in-place** such that each unique element appears only once.

Return:

number of unique elements

The first k elements of the array should contain unique values.

In simple terms:

- Remove repeated values
- Keep only one occurrence
- Modify the original array
- Return count of unique elements

3. Examples (Input → Output)

Example 1

Input:

nums=[1,1,2]

Output:

2

Modified array:

[1,2,_]

Explanation:

Unique elements:

1,2

Example 2

Input:

nums=[0,0,1,1,1,2,2,3,3,4]

Output:

5

Modified array:

[0,1,2,3,4,_,_,_,_,_]

Example 3

Input:

nums=[1]

Output:

1

4. Constraints

1 <= nums.length <=3*10^4

-100 <= nums[i] <=100

Special restrictions:

- Array is already sorted
- Must modify array in-place
- Use O(1) extra space

5. Core Concept (Pattern / Topic)

Two Pointers

6. Thought Process (Step-by-Step Explanation)

Brute Force

Create another array:

1. Traverse array
2. Store unique values separately
3. Copy back

Time:

O(n)

Space:

O(n)

Not optimal because extra memory is used.

Optimized Thinking

Observation:

Array is already sorted.

Therefore:

Duplicates are adjacent

Use two pointers:

- i → position of last unique element
- j → scans the array

Algorithm:

1. Start i=0
2. Traverse using j
3. If new value found:
    - Move i
    - Store current value at i
4. Return:

i+1

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

[0,0,1,1,1,2,2,3,3,4]

Process:

i

0 0 1 1 1 2 2 3 3 4

↑

j scans →

0 1 2 3 4

↑

Unique values move left

Final:

[0,1,2,3,4]

8. Pseudocode (Language Independent)

```
i=0

for j=1 to n-1:

  if nums[j]!=nums[i]:

    i++

    nums[i]=nums[j]
```

```
return i+1
```

9. Code Implementation

Python

```
class Solution:

    def removeDuplicates(
            self,
            nums):

        i=0

        for j in range(
                1,
                len(nums)
        ):

            if nums[j] != nums[i]:

                i += 1

                nums[i]=nums[j]

        return i+1
```

Java

```
class Solution {

    public int removeDuplicates(
            int[] nums) {

        int i=0;

        for(int j=1;
            j<nums.length;
            j++){
```

```
        if(nums[j]
              !=nums[i]){

           i++;

           nums[i]=
                 nums[j];
        }
     }

     return i+1;
  }
}
```

10. Time & Space Complexity

Time Complexity:

O(n)

Reason:

Single traversal of array.

Space Complexity:

O(1)

Reason:

Only pointer variables are used.

11. Common Mistakes / Edge Cases

1. Forgetting array is sorted

This approach works because:

duplicates are adjacent

2. Returning:

i

instead of:

i+1

3. Missing single element case

[1]

Output:

1

4. Creating unnecessary extra arrays

12. Detailed Dry Run (Step-by-Step Table)

Input:

[0,0,1,1,1,2,2,3,3,4]

j	nums[j]	i	Array
1	0	0	[0,0,1,1,1,2,2,3,3,4]
2	1	1	[0,1,1,1,1,2,2,3,3,4]
5	2	2	[0,1,2,1,1,2,2,3,3,4]
7	3	3	[0,1,2,3,1,2,2,3,3,4]
9	4	4	[0,1,2,3,4,2,2,3,3,4]

Return:

5

13. Common Use Cases (Real-Life / Interview)

Database record cleanup

Removing repeated logs

Data preprocessing

Duplicate filtering systems

14. Common Traps (Important!)

1. Using extra arrays
2. Returning wrong size
3. Missing sorted property
4. Incorrect pointer movement

15. Builds To (Related LeetCode Problems)

Progression:

- LC 26 → Remove Duplicates from Sorted Array
- LC 80 → Remove Duplicates from Sorted Array II
- LC 27 → Remove Element

- LC 283 → Move Zeroes
- LC 88 → Merge Sorted Array

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Extra Array	O(n)	O(n)	Additional memory
Set + Copy	O(n)	O(n)	Loses sorted benefit
Two Pointers	O(n)	O(1)	Optimal

17. Why This Solution Works (Short Intuition)

Since the array is already sorted, duplicates appear next to each other. The second pointer finds new unique elements, and the first pointer places them into the correct position.

18. Variations / Follow-Up Questions

1. Allow duplicates at most twice
2. Remove specific values
3. Remove duplicates from linked list
4. Count frequency of each element
5. Return unique elements instead of count
6.